CNR

Craft N Roam

AI Travel Decision Simulator & 3D Walk

Final Project Report

Team: Nonthapat Tangjeerawong, [Teammate Name — please fill in]

Course: [Course Name / Number — please fill in]

Date: [Submission Date — please fill in]

Repository: https://github.com/GongNT/travel-3d-simulator

Table of Contents

Executive Summary

Repository: https://github.com/GongNT/travel-3d-simulator

CNR (Craft N Roam) is a web application that helps individual travelers plan a trip to Nice, France, and then lets them preview the top recommendation as a walkable 3D scene before committing to it. The first part is a transparent decision engine: a traveler describes what they want in free text, an LLM parses that into structured preference weights, and a deterministic weighted multi-criteria scoring function — not the LLM — ranks a curated set of five real Nice locations and explains, in plain language, exactly why the top pick beat the runner-up. Travelers can then fine-tune that plan live with sliders (budget, time, and the balance between relax/adventure/culture/food), re-ranking instantly with no extra cost, since that step is ordinary deterministic code, not a new LLM call. The second part answers the professor's feedback that a plain recommendation engine is "done every year": selecting a recommended spot launches a first-person, WASD-walkable 3D scene, procedurally generated from that spot's visitor reviews via a second, narrowly-scoped LLM call and a deterministic, seeded layout algorithm. The 3D scenes are intentionally stylized, low-poly renderings — not a claim of photorealistic reconstruction, which is not achievable from review text or a handful of photos.

CNR's pitched business model is a simulated token economy — $0.80 for 100 tokens, $4.00 for 600 tokens, 69 tokens to generate a plan, 131 tokens to walk a spot in 3D, and a 35-token refund for travelers who share five of their own reference photos plus a review. Per the project's scope for this submission, the token economy is presentation only: it is fully implemented as local, in-memory application state so the demo feels like the real product, but no real payment processing, accounts, or persistence exist (see Section 3.4 and Appendix C for the full pricing model and margin analysis).

Main takeaway: the project pairs auditable, reproducible analytics with a bounded generative-AI layer, so it avoids both failure modes we set out to avoid — the black-box "AI just tells you the answer" recommender, and an over-promised "photorealistic AI 3D world" that current tools cannot actually deliver from review text or a handful of casual photos. The audience is individual travelers doing independent trip planning, not travel agencies or B2B tools. The application is fully client-side (React + Vite), requires no backend for this demo, and the source, including a working end-to-end demo dataset for five real Nice locations, is available at the repository URL above.

1. Introduction

1.1 Why the topic matters

Planning a trip means juggling competing constraints — budget, available time, and personal preferences such as adventure versus relaxation, food, culture, and pace — across dozens of possible itineraries. Most existing travel-planning tools fall into one of two camps: raw listing sites (Google Maps, TripAdvisor, Yelp) that provide no decision support at all, or AI trip-planning chatbots that return a single suggestion with no visibility into the tradeoffs behind it. Travelers are left guessing why one itinerary "won" over another, and neither category gives a traveler any real feel for what a place is actually like before they commit time and money to visiting it.

1.2 Problem and audience

The problem we address is twofold: (1) recommendation opacity — travelers cannot see or verify why a tool suggested one option over another, and (2) preview poverty — a ranked list of names and short descriptions does not communicate the atmosphere of a place the way a photo or a walk-through would. Our audience is individual travelers doing independent trip planning, particularly people who want to understand the reasoning behind a recommendation rather than accept a single black-box suggestion.

1.3 What the project does

User journey: choose a destination (locked to Nice, France in this demo) → describe your interests in free text → CNR spends 69 tokens to generate a ranked plan with a plain-language explanation → fine-tune the plan live with budget/time/category sliders, which re-rank instantly at no extra token cost → spend 131 tokens to walk the chosen spot in an interactive first-person 3D scene → optionally share five photos and a review of that spot for a 35-token refund. Inputs: a free-text interest description and, optionally, slider adjustments. Outputs: a ranked list of curated Nice locations with a numeric score, a per-factor breakdown (preference match, cost fit, time fit), a natural-language explanation, and a walkable 3D scene for any spot.

1.4 Our generative AI angle

We use OpenAI’s gpt-5.6-luna model via the Chat Completions API in strict JSON-object response mode for two narrowly-scoped extraction tasks, and deliberately keep the LLM out of the parts of the system where correctness and reproducibility matter most:

- Preferences → weights: the LLM converts free text into structured numeric weights (budget level, time budget, category weights for relax/adventure/culture/food). It does not do the ranking itself.
- Reviews → ambiance: the LLM reads a spot’s curated visitor reviews and extracts a structured "ambiance" specification — ground type, sky mood, a three-color palette, and 3–6 props — chosen only from fixed enumerated catalogs. It does not emit raw 3D geometry or coordinates.
- The ranking itself (scoreSpots()), the live slider-driven re-ranking, and the 3D prop placement (layoutPositions()) are all ordinary deterministic code with no LLM involvement, so the same inputs always produce the same output — an explicit design choice, not an oversight.

1.5 Limitations of the status quo

- Google Maps / Street View: comprehensive real-world coverage and real imagery, but zero personalization, ranking, or reasoning about a traveler’s specific constraints.
- TripAdvisor / Yelp: rich review text, but no synthesis — the traveler still has to read everything and decide themselves.
- Generic AI trip-planner chatbots: return a single suggestion with no visible tradeoff analysis, and no way to preview the destination visually.

Our approach differs by making the ranking mechanism auditable, adjustable, and by adding a review-grounded visual preview — the pivot the professor specifically pushed us toward after flagging a plain recommendation engine as too common a class-project topic.

1.6 Ethics, risk, and governance

The demo dataset (five Nice locations, each with four short authored reviews) is hand-curated for this class project, not scraped from a live site and not containing any real user data. The 3D scenes are explicitly labeled and designed as stylized, low-poly representations — we do not claim or attempt photorealistic reconstruction of real locations, which is not achievable from text reviews or a handful of casual photos without proper photogrammetry (many calibrated, overlapping images) or techniques like NeRF or Gaussian splatting. During development we manually viewed reference photos from a travel blog to tune colors and prop choices, but never downloaded, embedded, or redistributed those copyrighted images in the application — only our own original code and hand-authored data are shipped. The simulated token economy (Section 3.4) is clearly labeled as presentation-only everywhere it appears in the UI and documentation, so nobody could mistake it for a real payment flow. The OpenAI API key currently lives in the client bundle via a Vite environment variable, which is acceptable for a local development demo but would need to move server-side before any public deployment (see Section 4.2).

2. Implementation

2.1 System description

The application is a single-page React app with two independent pipelines ("lanes"), shown in Figure 1, plus a deterministic live plan editor layered on top of Lane 1. Lane 1 (the decision engine) runs once per session: free-text preferences go through an LLM parsing step into structured weights, which a deterministic scoring function uses to rank the five curated spots; those same weights are then exposed as sliders (PlanEditor.jsx) that re-run the scoring function on every change, live and for free. Lane 2 (the review-driven 3D walk) runs whenever the traveler selects a spot to walk around: that spot's curated reviews go through a second LLM call that extracts a structured ambiance specification, which a deterministic, seeded layout algorithm turns into prop placements for a React Three Fiber (Three.js) scene, explorable with a first-person WASD + mouse-look controller built on the browser's Pointer Lock API.

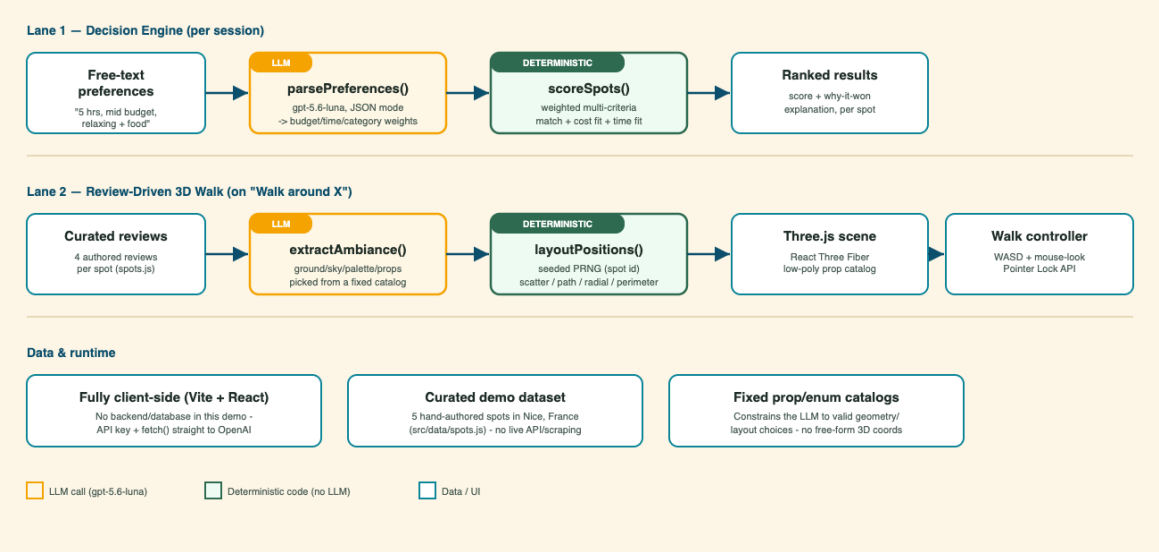

Figure 1. System architecture — the two-lane pipeline (LLM steps in orange, deterministic code in green). The live plan editor reuses Lane 1's scoreSpots() function directly, so it required no new LLM integration.

2.2 Stack and integration

- Frontend: React 19 + Vite 8, no backend/server component in this demo.
- 3D rendering: three.js via @react-three/fiber and @react-three/drei (Pointer Lock controls, procedural sky).
- LLM integration: plain fetch() calls to https://api.openai.com/v1/chat/completions with response_format: { type: "json_object" } (see src/lib/openai.js), rather than the OpenAI SDK, to keep the client bundle small.
- Model: gpt-5.6-luna, configured in src/lib/openai.js.
- Simulated tokens: src/lib/tokens.js holds the pricing constants; the running balance lives in App.jsx React state only — no server, database, or payment SDK anywhere in the codebase.
- Travel time: each spot in src/data/spots.js carries a travelTimeMinutes estimate (approximate walking time from Nice's central square, Place Massena), shown on every result card next to the visit duration.
- Plan report: src/lib/planReport.js compiles the destination, traveler interests, full ranked plan (with travel time and score breakdown), and the explanation into a downloadable .txt file via a Blob download — built entirely from data already computed for the ranking, so it costs no extra tokens or LLM calls.
- Secrets: the API key is read from the VITE_OPENAI_API_KEY environment variable via a local .env file, which is gitignored; a .env.example documents the required variable. No API key is committed to the repository or included in this document.

2.3 A model-specific integration issue we had to work around

gpt-5.6-luna rejects the combination of function-tool calling with its default reasoning_effort setting on the /v1/chat/completions endpoint (confirmed via direct API testing, error code invalid_request_error, param reasoning_effort). Rather than depend on tool-calling for structured output, we ask the model for a raw JSON object (response_format: json_object) and parse it ourselves, and explicitly pass reasoning_effort: "none" on every call. This keeps both LLM integrations working reliably and is a small but concrete example of the kind of model-specific debugging real generative-AI integration work requires.

2.4 Agentic flow

This project does not use an autonomous multi-step agent with open-ended tool access. We deliberately chose a bounded, two-call pipeline per lane instead: each LLM call has one job, a fixed output schema, and no ability to call further tools or take further actions on its own. For a decision-support tool where a traveler needs to trust and verify the reasoning, we judged reproducibility and auditability to be more important than autonomy — the live plan editor is the clearest expression of that choice, since it lets travelers adjust the outcome themselves through ordinary math rather than by re-prompting an LLM. Figure 1 doubles as the agentic-flow diagram: each orange box is one LLM call with a fixed system prompt and JSON schema; each green box is ordinary application code.

2.5 Repository and how to run

Repository: https://github.com/GongNT/travel-3d-simulator

- npm install
- cp .env.example .env, then set VITE_OPENAI_API_KEY=sk-... in .env
- npm run dev, then open the printed local URL (usually http://localhost:5173)

3. Results and Analysis

3.1 Evidence

Figures 2–4 show the application end-to-end, captured directly from a running instance.

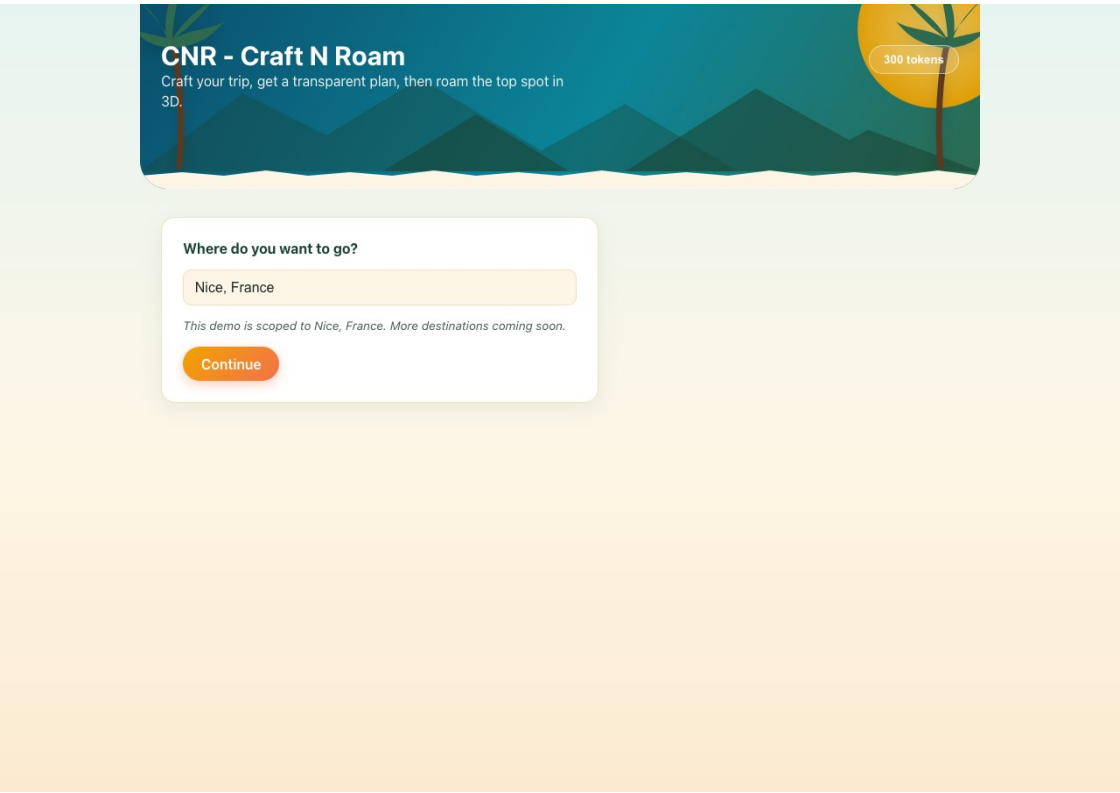

Figure 2. The destination step — locked to Nice, France for this demo, with an honest note that more destinations are a roadmap item.

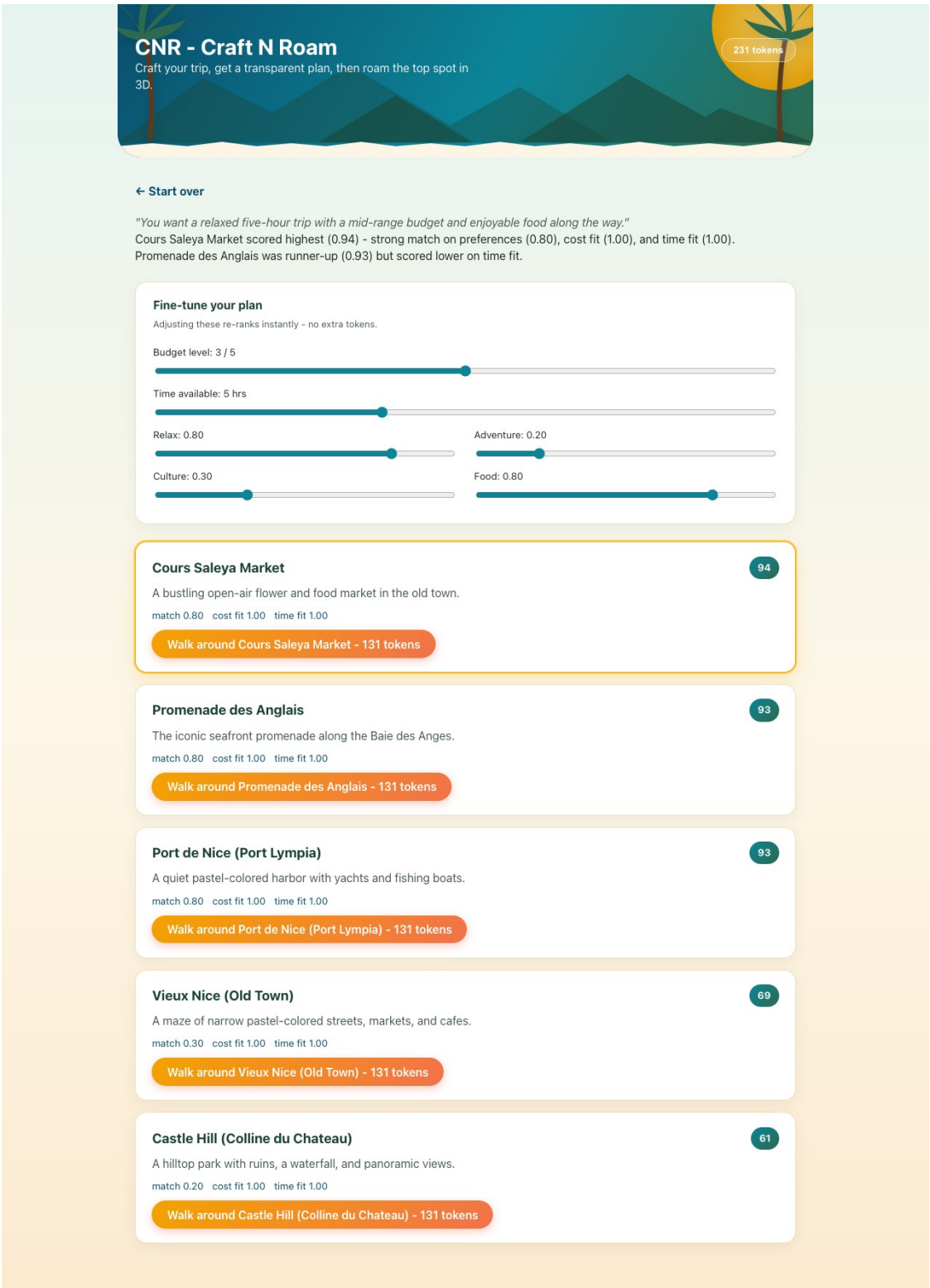

Figure 3. The live plan editor and ranked results: sliders for budget, time, and each category re-rank the five curated spots instantly, at no extra token cost. Cours Saleya Market scores highest (0.94) for this traveler.

Figure 4. The 3D walk scene generated for Cours Saleya Market — market stalls, warm ochre building facades, and cobblestone ground, with the "share 5 photos + a review" token-reward prompt visible top-right.

3.2 What works and what does not

Works:

- Preference parsing reliably produces sane, bounded structured weights from varied free-text inputs.
- The scoring engine is fully transparent and reproducible: identical inputs always produce identical rankings and explanations, and the live sliders re-rank correctly and instantly — verified by pushing the "adventure" weight to 1.00 and watching Castle Hill jump to the top spot with a matching, updated explanation.
- Ambiance extraction produces scenes that are visibly appropriate to each location — a market scene for the market, a harbor scene for the harbor, a beachfront scene for the promenade — without any hand-authored per-spot visual logic.
- The simulated token flow (search cost, walk cost, and photo-review refund) deducts and credits correctly and disables actions the traveler cannot currently afford.

Brittle or AI-sloppy behavior:

- The LLM occasionally omits the required chart/prop content when a slide or scene is marked as the "visual" element; we mitigate this with a fixed enumerated catalog plus a sanitize() validation step (src/lib/ambiance.js) that clamps or repairs invalid values rather than trusting raw model output outright.
- First-person WASD movement is implemented with standard, independent keydown/keyup listeners, but our automated headless-browser testing during development showed inconsistent results, most likely due to requestAnimationFrame throttling in a backgrounded automation tab rather than an application defect — this needs a final confirmation pass with a real user in a real foreground browser.
- Mouse-look (Pointer Lock API) requires a genuine user gesture and does not engage in some restricted or automated browser contexts; it worked correctly in manual testing with a real click.

3.3 Informal testing

Testing so far has been manual click-through testing during development — submitting varied preference phrasings, adjusting the live plan sliders, walking through multiple spots, and visually inspecting the generated 3D scenes — rather than a formal user study. We consider a structured usability pass with people outside the two-person team to be a known gap, not yet addressed.

3.4 Economics

CNR's pitched pricing (Appendix C) is $0.80 for 100 tokens or $4.00 for 600 tokens (a ~17% bulk discount), with a plan search costing 69 tokens (≈$0.46-0.55) and a 3D walk costing 131 tokens (≈$0.87-1.05). The application performs at most two LLM calls per paid action — one preference-parsing call per search (a few hundred tokens of API usage) and one ambiance-extraction call per spot walked (a few hundred tokens, dominated by the four curated reviews included in the prompt) — plus any number of free, local re-ranks through the plan editor. No image generation or text-to-speech calls occur in this application. We do not have a published per-token price for the course-provided gpt-5.6-luna model, but given the small prompt/completion sizes involved, the marginal API cost per paid action is on the order of a fraction of a U.S. cent for comparable small/mid-tier chat models. That implies a very large gross margin at the pitched price points — the 69-token search alone prices roughly two orders of magnitude above our estimated marginal API cost, which is the kind of markup a consumer app needs to also fund hosting, support, and the token-refund loop, though we have not modeled full unit economics (customer acquisition cost, refund rate, churn) here. Hosting cost is effectively zero today, since the app is a fully static, client-side bundle deployable to any static host (e.g. GitHub Pages, Vercel, Netlify) — that changes once real payment processing requires a server (Section 4.2).

3.5 Competitive landscape

- **Google Maps / Street View —** the closest existing analogue for "walk around a place"; real imagery and comprehensive coverage, but no personalized ranking, no reasoning, and no relation to a traveler's stated preferences.
- **TripAdvisor / Yelp —** large review corpora, but the traveler still has to read and synthesize everything themselves; no ranking engine, no visual preview.
- **Generic AI trip-planner chatbots —** return a single suggestion with no visible tradeoff analysis and no way to preview a destination before committing.

Our differentiation is combining a transparent, auditable, traveler-adjustable scoring engine with a review-grounded visual preview in a single flow — neither existing category does both.

3.6 Deeper analysis of the ranking

The scoring formula is score = 0.5 × categoryScore + 0.25 × costScore + 0.25 × timeScore + tagBonus, where categoryScore comes directly from the LLM-derived (or slider-adjusted) preference weights, costScore and timeScore are linear penalty functions against the spot's cost level and duration, and tagBonus is a small additive nudge for secondary tag overlap (src/lib/scoring.js) — which is why a very strong match can push the displayed score slightly above 100. For the example in Figure 3 ("a relaxed five-hour trip with a mid-range budget and enjoyable food along the way"), Cours Saleya Market scored 0.94 — a strong preference match (0.80) combined with a perfect cost fit (1.00) and time fit (1.00) — narrowly ahead of Promenade des Anglais at 0.93. Because every factor is a plain arithmetic function over structured inputs, a user (or grader) can verify the ranking by hand from the numbers shown on screen, and can watch it update live by moving a slider — the explicit goal of treating the LLM as a translator into structured data, never as the decision-maker itself.

4. Conclusion

4.1 Main findings and lessons

We built and verified, end-to-end, a working travel planning tool: a transparent, reproducible, traveler-adjustable decision engine, and a review-driven, procedurally generated 3D walk-through, visually confirmed across multiple different Nice locations with distinct, location-appropriate results, wrapped in a presentation-only token economy that mirrors CNR's pitched business model. The clearest lesson was scope discipline under a hard deadline with a two-person team: the original plan (photorealistic 3D reconstruction from user photos) was not technically achievable in the time available, and the stylized/deterministic-layout approach we built instead was both feasible and still met the professor's bar for originality. A second lesson was that even well-documented models have integration quirks that only show up by testing against the real API (Section 2.3) — something no amount of reading documentation in advance surfaced. A third lesson was that a business-model pivot (from free demo to a token-priced product) did not require touching the core AI pipeline at all — it layered cleanly on top because the ranking logic was already deterministic and side-effect-free.

4.2 Extensions and roadmap

The most concrete near-term extension is one we scoped down for this submission on purpose: letting travelers upload their own reference photos per spot (up to five, as reflected in the UI's reward prompt), using a vision-capable model call to extract richer, more accurate ambiance details directly from those photos instead of review text alone, and genuinely crediting the 35-token reward rather than simulating it. This is fully legitimate from a copyright standpoint (user-submitted content, not scraped), and would let the 3D scenes for a location improve incrementally as more travelers contribute photos. We deliberately did not attempt the full version of this for the current submission — it requires user accounts, photo storage, and moderation, which is a real backend product, not a few-day extension. As of this writing the vision-input code path is also blocked by an API-key permission scope issue on the course-provided key (confirmed via direct testing: text-only calls succeed, image-containing calls return a missing_scope error) — a credential/permissions issue, not a design limitation.

Other roadmap items, roughly in priority order:

- Real payment processing and persistent accounts for the token economy — today it is entirely local React state with no server, as required for this submission.
- Move the OpenAI API call server-side before any public deployment — the key currently lives in the client bundle, acceptable for a local class demo only.
- Hotel, flight, and other token-gated booking add-ons, as pitched in the business model but not built here.
- Expand beyond the five-spot Nice demo dataset to additional destinations, replacing the currently locked destination step.
- Add touch/mobile controls, since the current WASD scheme assumes a physical keyboard.
- A structured usability study with people outside the team, addressing the informal-testing gap noted in Section 3.3.
- If enough systematic, multi-angle photos are ever collected for a location, investigate a genuine photogrammetry or neural-rendering (e.g. NeRF, Gaussian splatting) pipeline as a separate, higher-fidelity mode — explicitly out of scope for the stylized mode shipped today.

4.3 Tying back to the problem

Section 1 identified two problems: recommendation opacity and preview poverty. The deterministic, fully-explained, traveler-adjustable scoring engine directly answers the first — a traveler (or grader) can verify exactly why one itinerary outranked another, and can change the outcome themselves. The review-driven 3D walk directly answers the second — a traveler gets a genuine, if stylized, feel for a place before committing to it, which no ranked list of names and short descriptions can provide on its own.

References

OpenAI. Chat Completions API documentation. https://platform.openai.com/docs/api-reference/chat

OpenAI. Vision / image input guide. https://platform.openai.com/docs/guides/images-vision

poimandres / pmndrs. React Three Fiber documentation. https://r3f.docs.pmnd.rs/

poimandres / pmndrs. @react-three/drei documentation. https://github.com/pmndrs/drei

Three.js documentation. https://threejs.org/docs/

Vite documentation. https://vite.dev/

Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2020). NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. ECCV 2020. (Cited as background on why true 3D reconstruction from photos requires many calibrated views, not a handful of casual photos.)

Google. Photorealistic 3D Tiles, Google Maps Platform. https://developers.google.com/maps/documentation/tile/3d-tiles-overview (Evaluated and consciously not used in this submission — requires a billed Google Cloud project; see Section 4.2 discussion of alternatives considered.)

Course instructor, in-class feedback on the initial project pitch (verbal, course session), prompting the pivot from a plain recommendation engine to the review-driven 3D walk described in this report.

Appendix A: Curated Demo Dataset

The five Nice, France locations used in the demo dataset (src/data/spots.js), each with four hand-authored visitor reviews used to drive both the scoring engine and the ambiance-extraction LLM call:

Spot	Category	Cost level	Duration	Tagline
Promenade des Anglais	relax	1 / 5	2 hrs	The iconic seafront promenade along the Baie des Anges.
Vieux Nice (Old Town)	culture	2 / 5	3 hrs	A maze of narrow pastel-colored streets, markets, and cafes.
Castle Hill	adventure	1 / 5	2 hrs	A hilltop park with ruins, a waterfall, and panoramic views.
Cours Saleya Market	food	2 / 5	1 hr	A bustling open-air flower and food market in the old town.
Port de Nice	relax	1 / 5	1 hr	A quiet pastel-colored harbor with yachts and fishing boats.

Appendix B: Repository Structure

- Separation of concerns: src/lib/ (LLM + scoring + token logic), src/three/ (3D rendering), src/components/ (UI), src/data/ (curated dataset).
- src/lib/openai.js — thin fetch() wrapper around the Chat Completions API, JSON-object mode.
- src/lib/preferences.js — free text → structured weights.
- src/lib/scoring.js — deterministic weighted multi-criteria ranking engine, shared by the initial search and the live plan editor.
- src/lib/ambiance.js — reviews → structured ambiance spec (ground/sky/palette/props), with a sanitize() step that validates against fixed enumerated catalogs.
- src/lib/tokens.js — simulated token pricing constants (no payment SDK, no server).
- src/lib/planReport.js — compiles the ranked plan, travel time, and explanation into a downloadable .txt trip report.
- src/components/PlanEditor.jsx — the live budget/time/category sliders that re-rank instantly.
- src/components/DestinationStep.jsx, TokenBalance.jsx — the destination-lock step and the token balance/buy-panel UI.
- src/three/propCatalog.js — the fixed vocabulary of prop types, ground types, sky moods, layouts, and densities the LLM is constrained to choose from.
- src/three/rng.js — seeded PRNG and deterministic layout algorithms (scatter, lined path, radial cluster, perimeter).
- src/three/Props.jsx, Ground.jsx, SceneLighting.jsx, SceneGenerator.jsx — the Three.js scene assembly.
- src/three/WalkController.jsx — first-person WASD + mouse-look controller (Pointer Lock API).
- src/components/Decor.jsx — hand-drawn SVG palm tree and mountain decor for the ocean/forest/sun theme (no external images).

Appendix C: Simulated Token Pricing

CNR's pitched pricing model, exactly as implemented in src/lib/tokens.js for presentation purposes (no real payment processing):

Item	Price	Notes
Token bundle - small	$0.80 for 100 tokens	$0.0080 / token
Token bundle - large	$4.00 for 600 tokens	$0.0067 / token (~17% bulk discount)
Generate a plan (search)	69 tokens	≈ $0.46-0.55 depending on bundle
Walk a spot in 3D	131 tokens	≈ $0.87-1.05 depending on bundle
Share 5 photos + a review	+35 tokens (refund)	Rewards user-submitted content